

JETGUARD

Requisitos Técnicos e Especificação Funcional

Documento de apresentação técnica para avaliação corporativa

Campo	Informação
Sistema	JetGuard
Tipo	Plataforma web de segurança patrimonial, operação, treinamentos, relatórios e análise de riscos
Versão base	1.1.9
Ambiente sugerido	Servidor Linux corporativo com PostgreSQL
Data	24/08/2026

1. Ideia do projeto

O JetGuard é uma plataforma web criada para centralizar processos de segurança patrimonial, operação, treinamentos, certificações, relatórios, análise de riscos, planos de ação, auditoria e acompanhamento de rotinas críticas em ambientes portuários, logísticos e corporativos.

A proposta é reduzir controles manuais, organizar evidências, padronizar registros, dar rastreabilidade às ações e permitir que gestores acompanhem indicadores e pendências em tempo real.

Finalidade central

Transformar rotinas operacionais, treinamentos, ocorrências, relatórios e controles de risco em fluxos digitais rastreáveis, auditáveis e acessíveis por perfil de usuário.

2. Módulos contemplados

Módulo	Descrição
Dashboard e indicadores	Visão executiva de números operacionais, pendências, registros, status, análises e acompanhamento por área.
Ocorrências e eventos	Registro, consulta, classificação, evidências, histórico, status, responsáveis, anexos e trilha de auditoria.
Relatórios	Relatórios operacionais e gerenciais com filtros, exportações e consolidação de dados relevantes.
Relatos de campo	Registro estruturado de informações observadas em operação, com apoio a evidências e acompanhamento.
Quadra de Segurança	Controle de posicionamento, movimentação, histórico, linha do tempo e dossiê em PDF.
Treinamentos	Treinamentos públicos e restritos, POCs, integração de motoristas, certificados, avaliação, assinatura e controle por CPF/e-mail.
Criador de treinamentos	Criação de treinamentos dinâmicos com etapas, perguntas, avaliação do usuário, grupos de acesso e geração de link.
Análise de riscos	Cadastro geral, análise completa, pontuações, dashboard, mapa de calor, controles, decisão e fechamento de ARC.
Plano de ação	Tratamento por fator de risco, responsáveis, mediadores, prazos, status, anexos e acompanhamento de conclusão.
Usuários e perfis	Perfis de acesso, grupos de treinamento, usuários internos, terceirizados e controle de permissões.

3. Módulo de treinamentos

- Treinamento de acesso ao Recinto Alfandegado, Integração de Motoristas, POC-SEP-006, POC-SEP-007 e treinamentos dinâmicos criados na plataforma.
- Controle de progresso por participante, CPF, e-mail, grupo de treinamento, etapa atual, aprovação, tentativas, assinatura e emissão de certificado.
- Certificados em PDF com código sequencial, token de validação, QR Code e página pública de validação.
- Envio de certificados por e-mail após conclusão, quando SMTP estiver configurado, sem bloquear a conclusão caso o envio falhe.
- Treinamentos restritos por grupos, permitindo que apenas colaboradores autorizados acessem o conteúdo pelo e-mail cadastrado.

4. Ocorrências, relatórios e operação

- Registro padronizado de ocorrências, eventos, evidências, responsáveis e histórico de alterações.
- Relatórios operacionais e gerenciais para apoiar auditorias, tomada de decisão e acompanhamento de pendências.

- Quadra de Segurança com movimentações, reposicionamento de contêineres, linha do tempo e dossiê em PDF.
- Controle de acesso por perfil para separar operação, administração, portaria, cadastro e supervisão.

5. Análise de riscos e planos de ação

- Cadastro de macroprocessos, setores, riscos, fatores de risco e controles preventivos, detectivos e corretivos.
- Criação de ARC com cálculo de probabilidade, consequência, risco inerente, avaliação residual e classificação.
- Dashboard com indicadores, mapas de calor, distribuição por nível, macroprocessos, decisões e situação dos planos.
- Plano de ação vinculado a cada fator de risco, com responsável, mediadores, prazo, status, anexos e histórico.
- Finalização da ARC somente após tratamento dos planos de ação vinculados, com decisão registrada e trilha de auditoria.

6. Requisitos técnicos da aplicação

Camada	Tecnologia / requisito
Frontend	Aplicação web em React, TypeScript e Vite, servida como arquivos estáticos por servidor web.
Backend	API Node.js com TypeScript, Express, Prisma ORM e rotas REST para os módulos do sistema.
Banco de dados	PostgreSQL em ambiente persistente, com schema gerenciado por migrations do Prisma.
Arquivos	Armazenamento persistente para uploads, anexos, evidências, certificados, PDFs e vídeos públicos.
Autenticação	Login com controle de perfis e permissões, com suporte recomendado a SSO corporativo.
E-mail	SMTP corporativo ou conta dedicada para envio de certificados e convites de treinamento.
Proxy reverso	Nginx ou Apache configurado para HTTPS, frontend estático, API, uploads e WebSocket, se aplicável.

7. Requisitos mínimos de infraestrutura

Item	Mínimo	Recomendado
Sistema operacional	Ubuntu Server 22.04 LTS	Ubuntu Server 24.04 LTS
CPU	2 vCPU	4 vCPU ou superior
Memória RAM	4 GB	8 GB ou superior
Armazenamento	40 GB SSD	80 GB SSD ou superior, com expansão conforme anexos e vídeos
Banco de dados	PostgreSQL 14+	PostgreSQL 15+ ou versão corporativa homologada
Rede	Acesso HTTPS público ou interno	DNS corporativo, firewall, TLS válido e monitoramento
Backup	Backup diário do banco e arquivos	Backup automatizado, retenção e teste de restauração

8. Autenticação corporativa e SSO

- Recomenda-se integrar o JetGuard ao provedor de identidade corporativo da empresa para login seguro por SSO.
- A integração pode considerar Microsoft Entra ID/Azure AD, Active Directory Federation Services, LDAP corporativo, SAML 2.0 ou OpenID Connect, conforme padrão adotado pelo TI.
- O acesso dos colaboradores deve priorizar a conta corporativa, reduzindo senhas locais e melhorando rastreabilidade, bloqueio e desligamento de usuários.
- Quando aplicável, o ambiente pode aproveitar o login da máquina/domínio corporativo para experiência de acesso integrada, respeitando as políticas de segurança da empresa.
- Perfis e permissões do JetGuard devem ser associados a grupos corporativos ou mantidos na aplicação, conforme diretriz do TI.

Requisito recomendado

SSO corporativo é recomendado para produção, especialmente em ambiente interno, para centralizar autenticação, MFA, bloqueio de usuários e auditoria de acesso.

9. Variáveis e configurações necessárias

Configuração	Finalidade
DATABASE_URL	String de conexão PostgreSQL usada pela API e Prisma.
PORT	Porta interna da API, por exemplo 3003.
JWT_SECRET / JWT_EXPIRES_IN	Segredo e tempo de expiração dos tokens de autenticação.
CORS_ORIGIN	Domínio autorizado para consumir a API.
PUBLIC_APP_URL / APP_URL	URL pública usada em links de certificados, validação e treinamentos.
SSO_CLIENT_ID, SSO_CLIENT_SECRET, SSO_ISSUER_URL	Variáveis recomendadas caso a empresa opte por SSO via OIDC/SAML.
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_FROM	Configuração para envio de e-mails pelo sistema.
UPLOAD_MAX_FILE_SIZE	Limite de tamanho para anexos, evidências e uploads.

10. Versionamento no GitHub

- O código-fonte deve ser mantido em repositório GitHub controlado pela empresa ou por repositório autorizado pela área de TI.
- O versionamento deve usar branch principal estável e branches de desenvolvimento para alterações, correções e novas funcionalidades.
- Cada atualização deve ser registrada em commits descritivos, permitindo rastreabilidade técnica das mudanças aplicadas.
- Recomenda-se fluxo com pull request, revisão, histórico de versões e tag para releases implantados em produção.
- Arquivos sensíveis não devem ser versionados: .env, bancos locais, backups, uploads, certificados privados, chaves e dumps de banco.

Importante

O GitHub deve armazenar o código e o histórico técnico do projeto. Senhas, variáveis de ambiente, certificados, uploads e backups devem permanecer fora do repositório.

11. Requisitos de segurança

- HTTPS obrigatório para acesso externo ou interno corporativo.
- Firewall liberando somente portas necessárias, como 80, 443 e acesso administrativo restrito.
- Banco de dados com usuário específico da aplicação e senha forte.
- Segredos e senhas armazenados em variáveis de ambiente, nunca no código-fonte.
- Autenticação segura com política corporativa, preferencialmente com SSO, MFA e bloqueio centralizado.
- Rotina de backup do PostgreSQL e dos diretórios persistentes.
- Controle de acesso por perfil: super_admin, administrador, analista, operador, portaria, cadastro e demais perfis definidos.
- Logs de aplicação, auditoria de ações críticas e monitoramento de falhas.

12. Dados persistentes

Para evitar perda de dados, estes itens devem permanecer persistentes entre atualizações:

- Banco PostgreSQL da aplicação.
- Diretório de uploads, evidências e anexos.
- Diretório de vídeos públicos utilizados nos treinamentos.
- Certificados e PDFs gerados, quando armazenados em disco.
- Backups automáticos e manuais.
- Arquivo de variáveis de ambiente do backend.

13. Requisitos de implantação e operação

Necessidade	Descrição
Servidor	Servidor Linux com acesso administrativo para instalação, configuração e manutenção.
Domínio	DNS apontado para o servidor, por exemplo security.movecta.com.br.
Banco	PostgreSQL criado com banco, usuário e senha específicos para o JetGuard.
Proxy	Servidor web configurado para publicar frontend, API, uploads e certificados.
SSO	Dados do provedor de identidade corporativo, caso a autenticação seja integrada à conta da empresa.
E-mail	Conta SMTP dedicada para envio de convites, certificados e notificações.
Backups	Rotina documentada para backup e restauração do banco e arquivos persistentes.
Atualização	Procedimento controlado com GitHub, build do frontend, atualização do backend, migrations e reinício da API.

14. Informações necessárias para disponibilização

- Servidor Linux homologado pela empresa.
- Domínio ou subdomínio apontado para o servidor.
- Acesso administrativo ou usuário técnico para implantação.
- Banco PostgreSQL com credenciais da aplicação.
- Conta de e-mail SMTP para envio de mensagens.
- Informações do provedor SSO corporativo, se a empresa optar por login integrado.
- Política de backup e local de armazenamento persistente.
- Repositório GitHub definido para versionamento e atualização do código.
- Perfis e usuários iniciais que terão acesso administrativo.